

sLiSCP: Simeck-based Permutations for Lightweight Sponge Cryptographic Primitives

Riham AlTawy, Raghvendra Rohit, Morgan He, Kalikinkar Mandal,
Gangqiang Yang, and Guang Gong

Department of Electrical and Computer Engineering, University of Waterloo,
Waterloo, Ontario, N2L 3G1, CANADA.

Abstract. In this paper, we propose a family of lightweight cryptographic permutations, named sLiSCP, with the sole aim to provide a realistic *minimal design* that suits a variety of lightweight device applications. More precisely, we argue that for such devices the area dedicated for security purposes should not only be consumed by an encryption or hashing algorithm, but also be used to provide as many cryptographic functionalities as possible. Our main contribution is the design of a lightweight permutation employing a 4-subblock Type-2 Generalized Feistel-like Structure (GFS) and round-reduced unkeyed Simeck with either 48 or 64-bit block length as the two round functions, thus resulting in two lightweight instances of the permutation, sLiSCP-192 and sLiSCP-256. We leverage the extensive security analysis on both Simeck (Simon-like functions) and Type-2 GFSs and present bounds against differential and linear cryptanalysis. Moreover, we analyze sLiSCP against a wide range of distinguishing attacks, and accordingly, claim that there exist no structural distinguishers for sLiSCP with a complexity below $2^{b/2}$ where b is the state size. We demonstrate how sLiSCP can be used as a unified round function in the duplex sponge construction to build (authenticated) encryption and hashing functionalities. The parallel hardware implementation area of the unified duplex mode of sLiSCP-192 (resp. sLiSCP-256) in CMOS 65 nm ASIC is 2289 (resp. 3039) GEs with a throughput of 29.62 (resp. 44.44) kbps.

Keywords: Lightweight cryptography, Cryptographic permutation, Simeck block cipher, Generalized Feistel Structure, Sponge duplexing, Authenticated encryption, Hash function.

1 Introduction

The area of lightweight cryptography has been investigated in the literature for over a decade, however, only recently NIST [45] has initiated a standardization project in response to the lack of standards that suit the bursting variety of constrained applications. In fact, long before NIST’s lightweight cryptography project [45], the cryptographic community has, in an ad-hoc manner, tried to establish some common criteria on how to define a lightweight cryptographic design (e.g., 2000 GEs for hardware area) [4, 38]. Nevertheless, such criteria

are rather generic, and specifically the established bound on the hardware area represents an upper bound for a passive RFID tag which may contain a total of between 1000 and 10000 GEs, out of which, a maximum of 20% is to be used for all security functionalities [38]. Other metrics include latency which maybe considered of a paramount importance for some applications such as automotive embedded systems that require a fast response time. However, for other highly resource-constrained applications (e.g., EPC tags), latency can be relaxed so that smaller area is realized. What remains the most important aspect in an acceptable realistic secure lightweight cryptographic design is its hardware footprint given that it offers acceptable metrics for throughput and latency.

Over the last decade, numerous symmetric primitives such as block ciphers, stream ciphers and hash functions have been proposed to secure resource-constrained applications. Examples of block ciphers include TEA [53], KATAN/KTANTAN [30], LED [35], PRESENT [24], HIGHT [37], EPCBC [56], TWINE [49], PRINCE [27], SIMON and SPECK [9], SIMECK [55], and SKINNY [10], lightweight hash function examples include PHOTON [34], QUARK [5], and SPONGENT [23], and lightweight stream cipher examples encompass Grain-128 [36], Trivium [29], MICKY [8], and WG [46]. These proposals aim to achieve hardware efficiency by adopting efficient round or feedback functions so that the targeted cryptographic functionality is provided while guaranteeing its security. However, none of these proposals has considered providing multiple cryptographic functions with low overhead, which might be a determining factor for its realistic adoption in many constrained devices. In other words, it is reasonable to assume that the available hardware area dedicated for security purposes should be used to provide encryption, authentication, hash computation, and possibly pseudo-random bit generation, which are the basic functionalities required by security services or protocols. Similar to the advantage of having an encryption algorithm where both encryption and decryption use the same round function, the concept of *cryptographic minimal design* aims to unify one design for as many cryptographic functionalities as possible. As a trade-off for having a minimal design, some redundancy may be introduced and thus, latency and throughput of individual functionalities may not be optimized.

In recent years, various authenticated encryption (AE) schemes have been developed (e.g., during the CAESAR competition [28]). Of particular interest are NORX-16 [7] and Ketje-JR [11] as they have state sizes of 256 bits (2880 GEs) and 200 bits (1270 LUTs), respectively, and also the lightweight AE scheme Grain-128a (est. 2769.5 GEs) [2]. However, all the latter lightweight AE schemes are optimized (e.g., MonkeyDuplexing [16]) for authenticated encryption and not to be used as a hash function [16]. One can achieve a minimal design using the Keccak permutation family [17]. However, the smallest instance of the Keccak family is Keccak-200 whose implementation cost in the duplex mode is 4900 GEs for 130 nm ASIC [39]. Consequently, we believe that there is a need to explore the design space of secure lightweight cryptographic permutations which are suitable for unifying a cryptographic design with a minimal overhead of multiple cryptographic functionalities.

Our contributions. We aim for a hardware efficient and secure cryptographic permutation for a minimal design, thus our contributions are as follows:

- We design the sLiSCP family of permutations, which adopts two of the most efficient and extensively analyzed cryptographic structures, namely a 4-subblock Type-2 Generalized Feistel-like Structure (GFS) [47, 26], and a round-reduced unkeyed version of the Simeck encryption algorithm [55]. Specifically, the round function of Simeck is an independently parameterized hardware efficient version of the Simon round function [9] and has set a new record in terms of hardware efficiency and performance in various platforms. Moreover, Simeck, Simon and Simon-like variants have been extensively cryptanalyzed by the public cryptographic community [51, 1, 19, 41, 42, 44].
- We investigate the security of the sLiSCP permutation against a wide variety of distinguishing attacks. We use the SMT/SAT tool developed in [41] and develop a Mixed Integer Linear Programming (MILP) model to evaluate the bounds for the probabilities of the differential and linear distinguishers. The security of sLiSCP against the known attacks exploiting low bit diffusion is ensured by choosing the number of rounds to be three times the number of rounds required for achieving full bit diffusion, as proposed in Simpira V2 [33]. We claim that sLiSCP has no structural distinguishers with complexity less than $2^{b/2}$ where b is the state size. This kind of claim has been used in the setting of the security claims of the Keccak permutation [13] and Simpira V.2 [33].
- We demonstrate how to use the sLiSCP permutation to construct authenticated encryption and hash functions in the duplex sponge construction. Moreover, our ASIC implementation results in CMOS 65 nm show that the areas of the unified modes of sLiSCP-192 and sLiSCP-256 are 2289 GEs and 3039 GEs with a throughput of 29.62 and 44.44 kbps, respectively.

In the following Section, we present the general construction of the sLiSCP permutation and its two instances, the Simeck^u- m box and its cryptographic properties.

2 Specification of sLiSCP

In this section, we formally describe the sLiSCP permutation, illustrated specifications are provided in the full version of the paper [3]. The core algorithm of the sLiSCP permutation is built upon the Simeck cipher’s round function and a 4-subblock Type-2 GFS construction.

2.1 Description of Simeck^u- m

We use Simeck^u- m as a round function in the sLiSCP permutation. Simeck^u- m is derived from the Simeck cipher whose block length is equal to m bits and its round function is iterated for u rounds, where each round is given by: $h_i(x) = R_i(x_0, x_1) = (((x_0 \lll 5) \odot (x_0 \lll 0)) + (x_0 \lll 1) + x_1 + rk_i, x_0)$, where $x = x_0 \| x_1$, $h_i : \mathbb{F}_2^m \rightarrow \mathbb{F}_2^m$, $\lll$ is a left cyclic shift operator, x_0 and x_1 are $\frac{m}{2}$ -bit words, rk_i is a $\frac{m}{2}$ -bit round key added at the i -th round and $+$ denotes the bitwise XOR in $\mathbb{F}_2^m$. We modify the round function as follows; instead of

adding a round key in $h_i, 0 \leq i \leq u-1$, we add a round constant rc_i in h_i where $rc_i = (C|t_i), C = (2^{\frac{m}{2}} - 2), t_i \in \mathbb{F}_2$ and $C|t_i$ denotes the bitwise OR between C and t_i . Let t be the integer representation of the u -tuple $(t_0, t_1, \dots, t_{u-1})$. Simeck ^{u} - m is defined as, $\text{Simeck}^{u-m}(x) = h_{u-1} \circ h_{u-2} \circ \dots \circ h_0(x)$, where the round constant rc_i is used in h_i at the i -th round. The round constants are generated using the LFSR described in Section 2.4. We, henceforth, refer to Simeck ^{u} - m as h_t^u . In sLiSCP, we choose all the round functions of GFS to be Simeck ^{u} - m and we consider it as an Sbox to systematically analyze our proposed permutation.

Definition 1 (Simeck ^{u} - m box) *A Simeck ^{u} - m box is a permutation of m -bit input constructed by iterating the Simeck- m cipher round function for u rounds with round constant addition in place of key addition. The nonlinear operation of such an Sbox is provided by iterating a simple AND operation followed by bitwise shifts and XORs for u rounds.*

2.2 Cryptographic Properties of Simeck ^{u} - m

We first define some notations which are used in this section. For a vectorial boolean function $f : \mathbb{F}_2^m \rightarrow \mathbb{F}_2^m$ with input (resp. output) difference δ_{in} (resp. δ_{out}), the differential probability, denoted by $\Pr(\delta_{in} \rightarrow \delta_{out})$, is defined as $\Pr(\delta_{in} \rightarrow \delta_{out}) = \frac{|\{x | f(x) \oplus f(x \oplus \delta_{in}) = \delta_{out}\}|}{2^m}$. The squared correlation of f with input mask Γ_{in} and output mask Γ_{out} is defined by $C^2(\Gamma_{in} \rightarrow \Gamma_{out}) = \left(\frac{\tilde{f}(\Gamma_{in}, \Gamma_{out})}{2^m} \right)^2$, where $\tilde{f}(\Gamma_{in}, \Gamma_{out}) = \sum_{x \in \mathbb{F}_2^m} (-1)^{(\langle x, \Gamma_{in} \rangle \oplus \langle f(x), \Gamma_{out} \rangle)}$ where $\langle x, y \rangle$ denotes the inner

product between vectors x and y . Let f^u denote the u -fold iteration of f and Δ^u be the set of all differential characteristics of f^u with probability $p > 0$. For f^u , the maximum expected differential characteristic probability (MEDCP) is given by: $\text{MEDCP}(f^u) = \max_{(\delta_0 \rightarrow \dots \rightarrow \delta_u) \in \Delta^u} \prod_{j=0}^{u-1} \Pr(\delta_j \rightarrow \delta_{j+1})$, where $\Pr(\delta_j \rightarrow \delta_{j+1})$ is the expected differential probability of $\delta_j \rightarrow \delta_{j+1}$ over one round of the keyed function f , when the key is picked uniformly and independently at random [31, 40]. The maximum expected linear characteristic correlation (MELCC) can be defined analogously.

Estimating the maximum differential probability of Simeck ^{u} - m . In our permutations, we employ a modified Simeck-48 and Simeck-64 round functions for Simeck ^{u} -48 and Simeck ^{u} -64, respectively. Due to their large input sizes, it is infeasible to build their differential distribution tables and compute the exact maximum differential probability (MDP) values. Alternatively, we use the maximum estimated differential probability (MEDP) to provide better estimates for differential characteristics for the sLiSCP permutations using the techniques in [41]. Table 1 presents the $\log_2$ probabilities of maximum expected differential (MEDCP) and linear squared correlation characteristics (MELCC) for Simeck-48 and Simeck-64.

The maximum differential probability is usually approximated by the maximum probability of a differential characteristic where such a probability is commonly assumed to be associated with differentials from characteristics with the

Table 1: Probabilities of the maximum differential and linear characteristics for Simeck^u-*m*, where *m* = 48 and 64. The probabilities are given in the $\log_2(\cdot)$ scale. Squared correlation is used for linear characteristic, thus the duality between both probabilities.

Rounds <i>u</i>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	...	24
Simeck ^u - <i>m</i>	0	-2	-4	-6	-8	-12	-14	-18	-20	-24	-26	-30	-32	-36	-38	-44	-44	-46	...	-62

optimal probability [33, 31]. However, due to the differential effect, we try to further tighten it by extracting the differentials from all the optimal characteristics and get their corresponding probabilities, then we approximate the MDP of the Simeck^u-*m* box by the maximum probability among these differentials. For that, we get tighter (but not strong) bounds for the probabilities of differential and linear characteristics of the permutation than by using the Simeck^u-*m* maximum differential characteristic probability. For Simeck^u-*m*, the probability of a differential (δ_0, δ_u) is defined as $\Pr(\delta_0 \xrightarrow{h_t^u} \delta_u) = \sum_{i=w_{min}}^{w_{max}} s_i \cdot 2^{-i}$, where w_{min} and w_{max} denote the minimum and maximum $\log_2$ characteristic probabilities, respectively, and s_i indicates the number of characteristics having probabilities equal to 2^{-i} . Table 2 presents the MEDP of the Simeck^u-*m* box calculated by $\text{MEDP}(\text{Simeck}^u\text{-}m) = \max_{(\delta_0, \delta_u) \in \Delta^u} (\Pr(\delta_0 \xrightarrow{h_t^u} \delta_u))$, where *m* = 48 and 64, and Δ^u denotes the set of differentials associated with characteristics with maximum probabilities.

Table 2: Estimation for the MDP for Simeck^u-*m* for *m* = 48 and 64

Rounds (<i>u</i>)	1	2	3	4	5	6	7	8	9
MEDP (Simeck ^u - <i>m</i>)	0	-2	-4	-6	-8	-11.299	-13.298	-16.597	-18.595

Instantiating *u* in Simeck^u-*m*. Based on our analysis, we decided to choose *u* = 6 for Simeck^u-48 and *u* = 8 for Simeck^u-64 because, we found that if we opted for nine (resp. 11) rounds so that the full bit diffusion is achieved at one sub-block after one step, then we need five steps for the full bit diffusion at the state level (i.e every state bit depends on all input bits) and thus based on our design criteria, fifteen steps are required for the permutation. This results in $15 \times 9 = 135$ (resp. $15 \times 11 = 165$) Simeck rounds for the permutation. On the other hand, by having six (resp. 8) rounds for Simeck^u-*m*, full bit diffusion is achieved at the state level in six steps, as a result, we only need eighteen steps for the permutation. Hence, sLiSCP employing Simeck-48 requires $18 \times 6 = 108$ rounds and sLiSCP employing Simeck-64 requires $18 \times 8 = 144$ rounds. Additionally, it can be seen in Table 1 that for Simeck-48 (resp. Simeck-64), the optimal characteristic probability decreases by a factor of 16 between 5 rounds and 6 rounds (resp. between 7 and 8 rounds) and by a factor of only 4 between 6 and 7 rounds (resp. between 8 and 9 rounds), which further enhances the resistance of Simeck^u-*m* parametrized by our round choices against differential and linear attacks. To achieve a better throughput and good resistance against

differential and linear cryptanalysis, we opted for $u = 6$ rounds and $u = 8$ rounds for Simeck^u-48 and Simeck^u-64, respectively.

For both Simeck⁶-48 and Simeck⁸-64, we considered all $(\delta_{in}, \delta_{out})$ differential pairs with $wt(\delta_{in}) = wt(\delta_{out}) = 1$ and we found that the probability of such differentials is zero, which means that both Sboxes have a branching number larger than 2. However, calculating the exact branch number is infeasible due to the large input size of Simeck^u- m . Using a tweaked method based on the propagation of the division property [50], we have computed the algebraic degree of Simeck⁶-48 (resp. Simeck⁸-64) component functions and our experimental results show that half of the components has algebraic degree of 13 (resp. 27) and the other half has a degree of 19 (resp. 36).

2.3 The Permutation F

The sLiSCP permutation is an iterative permutation, denoted by sLiSCP- b , over $\mathbb{F}_2^b$ where $b = 4 \times m$ and m is even. The construction of sLiSCP is based on Simeck^u- m (denoted as h_t^u) and a Type-2 GFS construction. An architecture of the sLiSCP permutation is shown in Figure 1. Let $(X_0^0, X_1^0, X_2^0, X_3^0)$ and $(X_0^s, X_1^s, X_2^s, X_3^s)$ be the input and output to the s -step permutation F , respectively where $X_i^0, X_i^s \in \mathbb{F}_2^m, 0 \leq i \leq 3$. Let $f : \mathbb{F}_2^{4m} \rightarrow \mathbb{F}_2^{4m}$ denote the step function. Then the permutation F is defined in terms of f as $F(X_0^0, X_1^0, X_2^0, X_3^0) = f^s(X_0^0, X_1^0, X_2^0, X_3^0) = (X_0^s, X_1^s, X_2^s, X_3^s)$, where $(X_0^{j+1}, X_1^{j+1}, X_2^{j+1}, X_3^{j+1}) = f(X_0^j, X_1^j, X_2^j, X_3^j)$, for $0 \leq j \leq s-1$ and $f^0 = f$. The step function f at j -th step is defined as $f(X_0^j, X_1^j, X_2^j, X_3^j) = (X_1^j, h_{t'}^u(X_3^j) + X_2^j + (C' | SC_{2j+1}), X_3^j, h_t^u(X_1^j) + X_0^j + (C' | SC_{2j}))$, where $t = (RC_{12j}, \dots, RC_{12j+2(u-1)})$, $t' = (RC_{12j+1}, \dots, RC_{12j+2u+1})$, $C' = 2^m - 256$, $P|Q$ denotes the bitwise OR between P and Q , and h_i is given by $h_i(x_0, x_1) = (((x_0 \lll 5) \odot (x_0 \lll 0)) + (x_0 \lll 1) + x_1 + (C | \gamma_i), x_0), 0 \leq i \leq u-1$, where $C = 2^{\frac{m}{2}} - 2$ and $\gamma_i = t_i$ (resp. t'_i) for h_t^u (resp. $h_{t'}^u$). For the constants RC_i

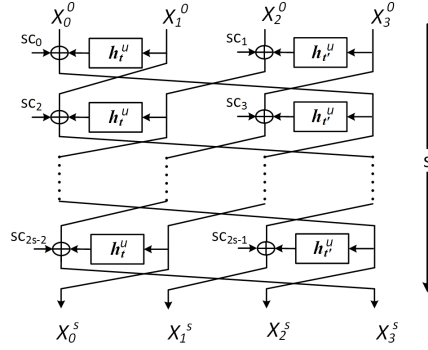


Fig. 1: sLiSCP permutation following 4-subblock Type-2 GFS using Simeck^u- m as h_t^u .

and SC_i generation, the reader is referred to Section 2.4. Table 3 presents the parameters for two lightweight instances of the sLiSCP permutation, which are denoted by sLiSCP-192 and sLiSCP-256.

Table 3: Parameter set for the 4 subblock GFS permutation F

Permutation (b -bit)	Subblock size m	Rounds u	Steps s	State size ($b = 4m$)	Total # rounds ($u \cdot s$)
sLiSCP-192	48	6	18	192	108
sLiSCP-256	64	8	18	256	144

2.4 Round and Step Constants

We add constants to Simeck ^{u} - m (round constants RC) and the GFS (step constants SC) to destroy the self-symmetry between rounds and steps. We use only one 6-stage LFSR (depicted in Figure 2) to generate both the round and step constants for sLiSCP-192. The j -th step round constants for h_t^6 and $h_{t'}^6$ are given by RC_{12j+2i} and $RC_{12j+2i+1}$, $0 \leq i \leq 5$, respectively. We use four extra XORs to generate 6-bit step constants from the same parallel LFSR. At the 6-th clock cycle in each 6-th round at j -th step, the states for the LFSR are $(s_{12j+10}, s_{12j+11}, s_{12j+12}, s_{12j+13}, s_{12j+14}, s_{12j+15})$, we assign $(s_{12j+10}, s_{12j+12}, s_{12j+14})$ to SC_{2j} and $(s_{12j+11}, s_{12j+13}, s_{12j+15})$ to SC_{2j+1} . For the other three values of SC_{2j} ($s_{12j+16}, s_{12j+18}, s_{12j+20}$) and SC_{2j+1} ($s_{12j+17}, s_{12j+19}, s_{12j+21}$), we use the following equations: $s_{12j+16} = s_{12j+10} \oplus s_{12j+11}$, $s_{12j+17} = s_{12j+11} \oplus s_{12j+12}$, $s_{12j+18} = s_{12j+12} \oplus s_{12j+13}$, $s_{12j+19} = s_{12j+13} \oplus s_{12j+14}$, $s_{12j+20} = s_{12j+14} \oplus s_{12j+15}$, $s_{12j+21} = s_{12j+15} \oplus s_{12j+16}$, $j \geq 0$. Thus, $SC_{2j} = s_{12j+10} || s_{12j+12} || s_{12j+14} || s_{12j+16} || s_{12j+18} || s_{12j+20}$ and $SC_{2j+1} = s_{12j+11} || s_{12j+13} || s_{12j+15} || s_{12j+17} || s_{12j+19} || s_{12j+21}$. The entire architecture of the parallel LFSR with feedback function $x^6 + x + 1$ to generate the j -th step round and step constants is shown in Figure 2. The constants for sLiSCP-256 are generated analogously using a 7-stage LFSR with primitive polynomial $x^7 + x + 1$. Further details are provided in the full paper [3].

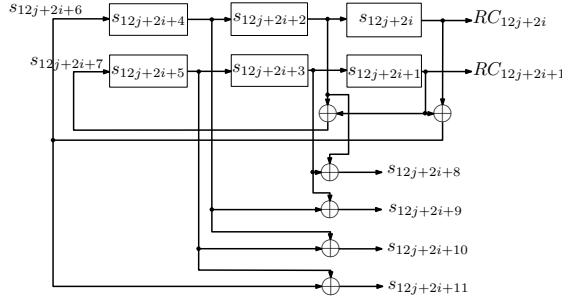


Fig. 2: Round and step constants generated by parallel LFSR at i -th round of j -th step.

3 Security Analysis

In this section, we analyze the security of the sLiSCP permutation by assessing its behavior against various distinguishing attacks. As the design of the sLiSCP permutation is based on the Simeck ^{u} - m box and the Type-2 GFS, the permu-

tation is analyzed against distinguishing attacks targeting Type-2 GFS designs when the employed round function is a large Sbox with specific differential and linear properties.

3.1 Differential and Linear Cryptanalysis

We assess the security of the sLiSCP permutation (denoted by F) by evaluating the lower bounds on the probabilities of the differential and linear characteristics. We use the terms characteristic and trail interchangeably throughout this section.

Bounding trails using the Wide Trail Strategy (WTS). To evaluate the lower bounds on differential and linear characteristics of sLiSCP, we first follow the WTS and compute the minimum number of active Simeck ^{u} - m boxes (h_t^u). In such a context, a Simeck ^{u} - m box is referred to as active if a non zero difference or a linear mask is presented at its input. Table 4 presents the lower bounds on the minimum number of active h_t^u for up to 18 steps. The bounds presented in Table 4 are generated by running a Mixed Integer Linear Programming (MILP) optimizer on a model describing our design.

Table 4: Lower bounds on the number of active h_t^u with respect to the number of steps.

Step	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Min. # of active h_t^u	0	1	2	3	4	6	6	7	8	9	10	12	12	13	14	15	16	18

One can see that the number of active h_t^u cycles every six steps. Given the bounds on the differential and linear correlation probabilities which we derived in Table 2, we can now evaluate $MEDCP(f^s)$ and $MELCC(f^s)$ when the number of steps $s = 18$ in both sLiSCP-192 and sLiSCP-256 by:

$$\begin{aligned}
 MEDCP(\text{sLiSCP-192}) &= MEDP(\text{Simeck}^{6-48})^{18} = (2^{-11.299})^{18} = 2^{-203.382(1)} \\
 MEDCP(\text{sLiSCP-256}) &= MEDP(\text{Simeck}^{8-64})^{18} = (2^{-16.597})^{18} = 2^{-298.746}.
 \end{aligned}$$

The duality between linear and differential cryptanalysis enables us to similarly apply the same approach to compute a bound on the MELCC. Over 18 steps, the maximum expected linear characteristic (squared) correlation is upper bounded by $2^{-203.382}$ and $2^{-298.746}$ for sLiSCP-192 and sLiSCP-256, respectively.

Bounding trails using the Long Trail Strategy (LTS). While the above bounds are for a single optimistically found characteristic, we can argue that such a characteristic might not be valid as it is pointed out in [31, 40], having all the 18 active Simeck ^{u} - m boxes exhibiting the maximum differential probability transitions is not always valid. Additionally, Dinu *et al.* [31] presented the long trail strategy which states that better bounds on differential and linear characteristics probabilities may be given if the design allows the propagation of long uninterrupted trails instead of the wide ones used in WTS.

Definition 2 (*Long Trail for n Simeck ^{u} - m boxes*) A differential path that includes a single path passing through n successive Simeck ^{u} - m boxes with no other

paths branching in. Such a trail can be static where an output subblock of the linear layer is equal to one of its inputs, or a dynamic trail where an output subblock of the linear layer is the result of XORing one of its input non active subblock with the output of an active Simeck^u-m box.

Based on the above definition, the *MEDCP* of a given differential trail is bounded by the product of the *MEDCP*s of its component long trails. Given a differential trail that is decomposed into i trails, where $\{\kappa_i\}_{i \geq 1}$ denotes the set of their corresponding lengths, then, the probability (pd) of such trail is upper bounded by $pd \leq \prod_{i \geq 1} MEDCP(\text{Simeck}^{u\kappa_i-m})$. Indeed, if $ds(u)$ and $ls(u)$ denote the *MEDCP* and *MELCC* of Simeck^u-m box, then it can be observed from Table 1 that $ds(nu) \ll ds(u)^n$, and $ls(nu) \ll ls(u)^n$, thus tighter bounds may be obtained.

Decomposition of an optimum trail. We apply the trail decomposition process on a 6-step trail with only six active Simeck^u-m box which is the trail returned by the MILP solver corresponding to the minimum number of active Sboxes. As depicted in Figure 3, the whole differential trail covers all the colored Sboxes. In such a trail, the XORs that receive two active input subblocks (indicated by the dashed colored lines) are assumed to cancel them resulting in zero-difference subblocks which are marked by black colored lines. Using the LTS, we can decompose this trail into five long trails, out of them, four trails are static with length one (green, blue, purple and orange colored trails) and the remaining one is dynamic (red trail) and of length two. Accordingly, the *MEDCP* of this differential trail is evaluated by $MEDCP(\text{Simeck}^{u-m})^4 \times MEDCP(\text{Simeck}^{2u-m})$ and equals $(2^{-12})^4 \times 2^{-30} = 2^{-78}$ (resp. $(2^{-18})^4 \times 2^{-44} = 2^{-116}$) for sLiSCP-192 (resp. sLiSCP-256). The above *MEDCP* bounds are considerably lower than the previously anticipated 6-step bounds calculated using Eq. (1) (based on WTS), which are $(2^{-11.299})^6 \approx 2^{-67.794}$ (resp. $(2^{-16.597})^6 = 2^{-99.582}$) for sLiSCP-192 (resp. sLiSCP-256).

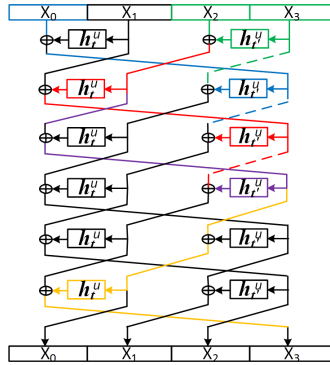


Fig. 3: A minimum number of active Simeck^u-m boxes trail decomposed into five long trails.

To this end, we claim that our analysis based on the minimum number of active Sboxes underestimates the security of sLiSCP with respect to differential and linear cryptanalysis, and that although the above analysis provide a provable bound against these types of attacks, they are not tight and as shown, tighter bounds may be provided using the LTS.

3.2 Meet/Miss in the Middle Distinguishers

Based on our design criteria, we require that the total number of steps is equal to three times the number of steps required for full bit diffusion. Consequently, splitting sLiSCP into two or even three parts always results in an intermediate state that achieves full bit-diffusion. More precisely, the evaluation of each bit in an intermediate state i requires the knowledge of the whole b bits of state $i - 6$. For that, the evaluation of an intermediate partial state has no advantage over the brute force guessing, thus we believe that sLiSCP is secure against these type of attacks. On the other hand, miss-in-the-middle distinguishers such as impossible differential (ID) and zero-correlation (ZC) distinguishers are a special kind of meet-in-the-middle distinguisher that exploits contradicting conditions at an intermediate state which makes matching such an intermediate state attributes impossible [18, 25]. Based on the results provided in [21, 42], we believe at most ten steps may be analyzed. Accordingly, having eighteen steps of sLiSCP- b is sufficient to mitigate distinguishers based on meet-and-miss-in-the-middle approaches. More detailed analysis about the distinguishers are provided in the full paper [3].

3.3 Integral and Zero-sum Distinguishers

The results in [22] imply the existence of 8-round integral distinguisher for sLiSCP. However, traditional integral attacks are less effective on bit-oriented ciphers which lead to the development of the new generalized division property [50]. Employing the algorithms proposed in [57, 54], we found nine-round division property distinguishers with data complexities 2^{188} and 2^{191} for sLiSCP-192. More analysis on sLiSCP-256 and Simeck ^{u} - m is presented in the full paper [3]. As for zero-sums, we have found 9-step distinguishers for both instances of sLiSCP.

3.4 Self Symmetry-based Distinguishers

To make the step functions in the GFS different, we add a 12-bit constant including step and round constants in each h_t^u . Our choice of LFSR ensures that each pair of such constants does not repeat due to the periodicity of the 6-tuple sequence constructed from the decimated m -sequence of period 63, which enables our design to avoid self symmetry-based distinguisher including slide [20], rotational and invariant subspace distinguishers [43, 48].

4 Applications of sLiSCP

The sLiSCP permutation is designed to be used in lightweight applications to provide as many cryptographic functionalities as possible. We use sLiSCP in the sponge framework to construct authenticated encryption (AE), stream cipher, MAC and hash function.

4.1 Why the Sponge Framework?

Sponge constructions are very diversified in terms of the offered security level, particularly, it is proven that the sponge and its single pass duplex mode offer a $2^{c/2}$ bound against generic attacks [12, 15]. However, for keyed modes such as MAC, stream cipher and authenticated encryption, a security level of 2^{c-a} is proven when the number of queries is upper bounded by 2^a [16]. When restricting the data complexity to a maximum of 2^a queries with $a < c$, one can reduce the capacity and increase the rate for a better throughput with the same security level. The sponge duplex AE mode requires the uniqueness of nonce when encrypting different messages with the same key as the ability of the attacker to obtain multiple combinations of input and output differences leaks information about the inner state bits which may lead to the reconstruction of the full state [15, 11]. One might exploit a nonce reuse differential attack. However, such an attack depends on the probability of the best differential characteristic and the number of rounds of the underlying permutation. If the permutation offers enough resistance against differential cryptanalysis, the feasibility of nonce reuse differential attack is minimal.

We use the sLiSCP mode which is a combined version of the duplex sponge mode used in ASCON [32, 16] and NORX [6] that realizes the objectives we aimed for. The objectives are: 1) Flexibility to adapt the same circuitry for both keyed and unkeyed functionalities in a hardware friendly manner; 2) High key agility that fits the lightweight requirements; 3) Simplicity for keeping the encryption and decryption algorithms similar; and 4) Plaintext and ciphertext blocks are generated online without processing the whole input and encrypted data first.

4.2 The sLiSCP Mode: AE and Hash

The initialization and finalization phases for the sLiSCP mode are derived by modifying the keyed initialization and finalization stages of the ASCON [32] and NORX [6]. The advantages of the adopted mode are that the initialization and finalization stages are more hardware efficient, and key recovery and tag forgery are hard even if an internal state is recovered. We also use the domain separation technique which is used in NORX as it runs for all rounds of each stage, and thus reduces the chances of side channel analysis and offers uniformity across different stages. The separation between the processing of different types of inputs is important to distinguish between the roles of the processed data. To this end, we have only one round function that incorporates absorption, squeezing, and domain separation.

Our sLiSCP permutation is based on the Type-2 GFS where there are four subblocks and each subblock is either 48 or 64 bits for sLiSCP-192 and sLiSCP-256, respectively. Since we use it in sponge-based modes, we need to specify exactly from where the r -bit input is absorbed and the r -bit output is squeezed. For sLiSCP permutations, we consider the b -bit state as a series of four m -bit subblocks, X_0, X_1, X_2 , and X_3 , where m is equal to 48 and 64 for sLiSCP-192 and sLiSCP-256, respectively. Each subblock X_i can be viewed as a series of $l = \frac{m}{8}$ bytes, $B_{li+0}, B_{li+1}, \dots, B_{li+(l-1)}$, thus the state S is divided into $L = 4l$

bytes and denoted by $S = (B_0, \dots, B_{L-1})$ where $L = 24$ and 32 for sLiSCP-192 and sLiSCP-256, respectively. We view the state S as $S = (S_r, S_c)$, where S_r and S_c denote the rate and capacity parts, respectively, S_r is defined as $S_r = (B_6, B_7, B_{18}, B_{19})$ for sLiSCP-192 and $S_r = (B_8, B_9, B_{10}, B_{11}, B_{24}, B_{25}, B_{26}, B_{27})$ for sLiSCP-256 and S_c consists of the remaining bytes.

Authenticated Encryption. An authenticated encryption ($\mathcal{AE}$) algorithm takes as input a secret key K , a nonce N , a block header A (a.k.a, associated data) and a message M and outputs a ciphertext C with $|C| = |M|$, and an authentication tag T . An authenticated encryption scheme is defined as $\mathcal{AE} : \{0, 1\}^k \times \{0, 1\}^n \times \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{0, 1\}^* \times \{0, 1\}^t$ with $\mathcal{AE}(K, N, A, M) = (C, T)$ where k is the bit length of K , n is the bit length of N and t is the tag size in bits. We denote an instance of sLiSCP in a keyed mode by sLiSCP- b/k , where b and k denote the state size and the key length, respectively. In such a mode, we limit the number of processed bits per key to 2^a such that one can attain bit security equal to 2^k when $c \geq k + a + 1$ [14]. There are three instances of AE based on sLiSCP, which are sLiSCP-192/80, sLiSCP-192/112, sLiSCP-256/128. The recommended parameters for sLiSCP when used in the AE mode are listed in Table 5.

Table 5: Recommended parameters for sLiSCP- b/k in the AE mode

Algorithm	Key	Nonce	Tag	Block size r	Capacity c	Usage exponent a
sLiSCP-192/80	80	80	80	32	160	72
sLiSCP-192/112	112	80	112	32	160	40
sLiSCP-256/128	128	128	128	64	192	56

Padding: Padding is necessary when the length of the processed data is not a multiple of the rate r value and also to act as a delimiter between data of unknown lengths. Since the keys are of fixed length, we need to pad it by appending zeros only if its length is not a multiple of r such that the padded K is divided into ℓ_K r -bit blocks $K_0 \| K_1 \| \dots \| K_{\ell_K-1}$. We use the padding rule (10*) denoting a single 1 followed by required 0's to the message M and associated data A such that their lengths after padding are multiple of r .

The sLiSCP AE algorithms: The sLiSCP AE scheme consists of initialization, processing associated data, encryption, decryption and finalization algorithms. We start by describing the key and nonce loading and the tag extraction phase. Initially, the state is loaded with a nonce N and a key K whose bytes are denoted by $N = (NB_0, \dots, NB_{\lceil \frac{n}{8} \rceil - 1})$, and $K = (KB_0, \dots, KB_{\lceil \frac{k}{8} \rceil - 1})$, respectively, and remaining bytes are set to zero (we refer to this procedure by *initialize*($\cdot, \cdot$)). Nonce bytes are divided and loaded in the even indexed subblocks, X_0 and X_2 , in an ascending byte order. The key is loaded in the odd indexed subblocks, X_1 and X_3 in the same manner, and if the key size is larger than half the state size, then remaining key bytes populate the the remaining bytes in X_0 and X_2 equally and in an ascending order. For example, a 112-bit key is loaded in sLiSCP-192 state as follows: $B_6 \dots B_{11} \leftarrow KB_0 \dots KB_5$, $B_{18} \dots B_{23} \leftarrow KB_7 \dots KB_{12}$, and $B_5, B_{17} \leftarrow KB_6, KB_{13}$. The tag extraction procedure, denoted by *tagextract*($\cdot$) extracts tag values from the same byte positions which are used in the key

initialization stage. Exact byte position assignments are provided in the full paper [3].

After applying the padding rule, the associated data A and message M are denoted by $A = A_0 \parallel \dots \parallel A_{\ell_A-1}$ and $M = M_0 \parallel \dots \parallel M_{\ell_M-1}$, resp., where each M_i and A_i are of r -bit block and ℓ_A and ℓ_M are the numbers of r -bit blocks for A and M . The ciphertext for message M is denoted by $C = C_0 \parallel \dots \parallel C_{\ell_M-1}$. The sLiSCP AE algorithms are described in Figure 4. The decryption procedure returns the message blocks $M_i, i = 0, 1, \dots, \ell_M - 1$, and it runs the same procedure as the encryption procedure, except that each $M_i = C_i \oplus S_r$, then S_r is replaced by C_i , and the last $M_{\ell_M-1} = \lfloor S_r \rfloor_{(|M| \bmod r)} \oplus C_{\ell_M-1}$, then S_r part is replaced by $C_{\ell_M-1} \parallel (\lceil S_r \rceil^{(r-|M| \bmod r)} \oplus (1 \parallel 0^{(r-1-|M| \bmod r)}))$. Decrypted messages are returned only if the calculated tag is the same as the received tag. We do not claim security in the event of nonce reuse, although, the initialization and finalization stages combined by the number of rounds used in the sLiSCP permutation tremendously reduces the effect of such attacks. We claim no security for reduced-round versions of the sLiSCP permutation operating in the sLiSCP modes. The security levels for data confidentiality, data integrity, associated data integrity and nonce data integrity are 80, 112 and 128 for sLiSCP-192/80, sLiSCP-192/112 and sLiSCP-256/128, respectively.

a Initialization algorithm 1: Input: Nonce N and Key K 2: Output: State S 3: $S \leftarrow F(\text{initialize}(N, K))$ 4: for i from 0 to $\ell_K - 2$ do 5: $S \leftarrow F((S_r \oplus K_i), S_c)$ 6: end for 7: $S \leftarrow F((S_r \oplus K_{\ell_K-1}), (S_c \oplus 0^{c-1} \parallel 1))$	b Processing A 1: Input: $A = A_0 \parallel \dots \parallel A_{\ell_A-1}$ and State S 2: Output: State S 3: for i from 0 to $\ell_A - 2$ do 4: $S \leftarrow F((S_r \oplus A_i), (S_c \oplus 0^{c-1} \parallel 1))$ 5: end for 6: $S \leftarrow F((S_r \oplus A_{\ell_A-1}), (S_c \oplus 0^{c-2} \parallel 2))$
c Encryption algorithm 1: Input: State S and Plaintext $M = M_0 \parallel \dots \parallel M_{\ell_M-1}$ 2: Output: Ciphertext $C = C_0 \parallel \dots \parallel C_{\ell_M-1}$ and State S 3: for i from 0 to $\ell_M - 2$ do 4: $C_i \leftarrow M_i \oplus S_r$ 5: $S \leftarrow F(C_i, (S_c \oplus 0^{c-2} \parallel 2))$ 6: end for 7: $C_{\ell_M-1} \leftarrow M_{\ell_M-1} \oplus S_r$ 8: $S \leftarrow F(C_{\ell_M-1}, (S_c \oplus 0^{c-3} \parallel 4))$ 9: $C_{\ell_M-1} \leftarrow \lfloor C_{\ell_M-1} \rfloor_{(M \bmod r)}$	d Finalization algorithm 1: Input: State S and Key K 2: Output: Tag T 3: for i from 0 to $\ell_K - 1$ do 4: $S \leftarrow F((S_r \oplus K_i), S_c)$ 5: end for 6: $T \leftarrow \text{tagextract}(S)$

Fig. 4: The sLiSCP AE algorithms

Hash Computation. A hash function takes as input a message M , and a standard initialization vector IV , and then returns a fixed size output H , called

hash or message digest. Mathematically, the hash function is defined as $\mathcal{H} : \{0,1\}^* \times \{0,1\}^{iv} \rightarrow \{0,1\}^h$ with $H = \mathcal{H}(M, IV)$ where iv is the length of the IV and h is the length of the hash in bits. Below we explain how to use sLiSCP to compute the hash of a message.

Absorbing and Squeezing: Before applying the absorbing phase, the state is first initialized with the IV in Table 6 similar to nonce initialization in the AE mode (cf. Sec. 6.4 in [3]), and the same padding rule (10*) is applied to the input message M to make the message length a multiple of r . Each message block is absorbed by XORing it to the S_r part of the state, then the sLiSCP permutation is applied. After absorbing all message blocks, the h -bit output is extracted from the S_r part of the state, outputting r' bits at a time, followed by applying the sLiSCP permutation, until a total of $\lceil h/r' \rceil$ extractions are completed. If the resulting extracted bits are more than the desired hash length, truncation is performed.

Preimage security: It has been shown that inverting the squeezing phase falls in the category of the multiblock CICO problem, which requires $2^{\min(h,b)-r}$ computations to recover the state before the squeezing phase [13, 34]. Once an internal state is recovered one can launch an MITM attack with $2^{c/2}$ computations to get a preimage of a given hash of length h . The latter condition reduces the generic preimage attack on the sponge-based hash functions from 2^h to $\min(2^{\min(h,b)}, \max(2^{\min(h,b)-r}, 2^{c/2}))$. Guo *et al.* [34] suggested using a flexible squeezing bit rate $r' < r$ that offers a trade off between speeding up the hash computation and preimage security. A smaller r' would make the time complexity of a preimage attack equal to $2^{h-r'}$.

We adopt a standard initialization vector, $IV = h/2 \| r \| r'$ where $h/2$, r , and r' are encoded using 8 bits, as used in PHOTON [34]. The security levels for the recommended parameters for sLiSCP in the hashing mode are given in Table 6.

Table 6: Parameters for sLiSCP- b in the hashing mode and their security levels

Algorithm	IV	h	r	r'	c	Collision	Sec. preimage	Preimage
sLiSCP-192	0x502020	160	32	32	160	80	80	128
sLiSCP-256	0x604040	192	64	64	192	96	96	128
sLiSCP-256	0x604020	192	64	32	192	96	96	160

5 Hardware Implementation and Results

We implement our sLiSCP permutation using the parallel hardware architecture which is shown and fully explained in the full version of the paper [3]. We use the same ASIC design flow and metrics as described in Simeck [55]. Our implementation results are based on STMicroelectronics CMOS 65nm CORE65LPLVT library and the areas are obtained before the place and route phase in order to compare fairly with other lightweight candidates. To keep the consistency with other sponge based primitives, the throughput is computed at a frequency of 100 kHz using the following formula: $Throughput = \frac{r'}{(u*s)} * 100$ kbps. Our implementation areas for sLiSCP-192 (resp. sLiSCP-256) permutation are 2153 (resp. 2833) GEs in 65 nm ASIC. By applying the sLiSCP permutations repeatedly, we carry two implementations for the hash and AE modes in order to contrast with other

dedicated designs. The implementations of these two modes involve the input of key, nonce, associated data, message, and the bit domain separator; and also involve the output of the ciphertext, hash value and tag. Our results for the hash and authenticated encryption modes of sLiSCP are presented in Table 7, as well as a comparison with other lightweight hash functions and AE algorithms. If a unified mode is used for both functionalities, then the consumed GE area will be dominated by that of the AE mode.

Table 7: Parallel hardware implementation of sLiSCP modes and comparison with other lightweight hash and AE primitives. Throughput is given for a frequency of 100 kHz.

Hash function	Parameters ^a				Security(bits)			Process	Latency	Area	Throughput
	r	c	r'	h	Pre	2nd Pre.	Coll.	(nm)	(Cycles)	(GEs)	(kbps)
sLiSCP-192	32	160	32	160	128	80	80	65	108	2271	29.62
Photon-160/36/36 [34]	36	160	36	160	124	80	80	180	180	2117	20.00
D-Quark [5]	16	160	16	176	160	80	80	180	88	2819	18.18
Spongnet-160/160/16 [23]	16	160	16	160	144	80	80	130	90	2190	17.78
Keccak- f [40,160] [39]	40	160	40	200	160	160	80	130	18	4900	222.22
Keccak- f [72,128] [39]	72	128	72	200	128	128	64	130	18	4900	400.00
sLiSCP-256	64	192	64	192	128	96	96	65	144	3019	44.44
sLiSCP-256	64	192	32	192	160	96	96	65	144	3019	22.22
Photon-224/32/32 [34]	32	224	32	224	192	112	112	180	204	2786	15.69
Spongnet-160/160/80 [23]	80	160	80	160	80	80	80	130	120	3139	66.67
Spongnet-224/224/16 [23]	16	224	16	224	208	112	112	130	120	2903	13.33
Spongnet-256/256/16 [23]	16	256	16	256	240	128	128	130	140	3281	11.43
S-Quark [5]	32	224	32	256	224	112	112	180	64	4640	50
AE algorithm	t				Con. ^b	Int. ^c					
sLiSCP-192/80	32	160	32	80	80	80	-	65	108	2289	29.62
sLiSCP-192/112	32	160	32	112	112	112	-	65	108	2289	29.62
sLiSCP-256/128	64	192	64	128	128	128	-	65	144	3039	44.44
Ketje-Jr [11]	16	184	16	96	96	96	-	-	-	4900 ^d	-
NORX-16 [7]	128	128	128	96	96	96	-	-	-	2880	-

^a r , c , r' , h and t denote the input bitrate, capacity, output bitrate, digest length and tag size, respectively.

^b Confidentiality of plaintext.

^c Integrity of plaintext, associated data and nonce.

^d Considering it uses Keccak-200 as its underlying permutation, its area is at least 4900 GEs.

Our implementation in CMOS 65 nm shows that the area for the hash mode of sLiSCP-192 (resp. sLiSCP-256) is 2271 (resp. 3019) GEs with a throughput of 29.62 (resp. 44.44 kbps or 22.22 kbps depending on r') kbps. When compared with other primitives with similar internal states, the area of sLiSCP-192 is slightly larger than that of the serialized implementation of Photon-160/36/36 and is comparable with that of Spongnet-160/160/16. However, the area of sLiSCP-192 is quite smaller than that of D-Quark, Keccak- f [40,160], Keccak- f [72,128], where the areas of Keccak- f [40,160] and Keccak- f [72,128] are permutations only. In terms of throughput, sLiSCP-192 is better than Photon-160/36/36, D-Quark, and Spongnet-160/160/16. The area of sLiSCP-256 is a little larger than that of the serialized result of Photon-224/32/32 and is comparable with that of Spongnet-160/160/80, Spongnet-224/224/16, Spongnet-

256/256/16, and is smaller than that of S-Quark. The relevant throughput is only smaller than that of Spong-160/160/80 and S-Quark.

For the authenticated encryption mode, the area of sLiSCP-192 (resp. sLiSCP-256) is 2289 (resp. 3039) GEs with a throughput of 29.62 (resp. 44.44) kbps. sLiSCP-256 has a GE area that is comparable with the estimated area of NORX-16, while sLiSCP-192 is quite smaller than NORX-16. Both areas of sLiSCP-192 and sLiSCP-256 are much smaller than that of Ketje-Jr. We note that serialized implementations of sLiSCP modes result in more savings in GE area and thus enable its adoption in highly constrained devices such as EPC tags. Overall, both the hash and authenticated encryption modes of sLiSCP are competitive with others in terms of area and throughput.

6 Concluding Remarks

In this section, we conclude the paper by highlighting some of the design decisions we have made for the construction of the sLiSCP permutation. Most of the contents of this section have been stated in a scattered way in earlier sections, so we aim by addressing these points again to reiterate some important conclusions that may have been missed by a reader.

- *Another sponge-based primitive:* We design sLiSCP in response to the noticeable shortage of lightweight secure cryptographic permutations which can be used in the sponge framework to provide a unified secure design which offers as many cryptographic functionalities as possible. In fact, most of the lightweight symmetric key primitives that exist in the literature are dedicated to offer a specific cryptographic functionality and accordingly are optimized as such. Other than Keccak-200 permutation which has a parallel hardware implementation cost of around 4900 GEs [39], we cannot find a lightweight cryptographic permutation. On the other hand, sLiSCP-192 in a duplex mode has a parallel implementation cost of 2289 GEs on a 65 nm ASIC, which enables its realistic adoption in constrained lightweight applications to provide a minimal cryptographic design.
- *Simeck is based on the generalized round function of NSA’s Simon:* The justifications of the parameters and design choices for Simon by the NSA remain unclear. However, Simeck is an independently parameterized unkeyed version of the generalized Simon round function. In addition to being fully analyzed by its designers [55] where all the parameter choices have been justified, Simeck has been publicly cryptanalyzed for over three years. Finally, Simeck offers one of the lowest hardware footprints which is even smaller than Simon’s.
- *The sLiSCP permutation is based on a GFS like the MD/SHA family of hash functions.* The MD/SHA family is a special instantiation of the Feistel construction which is vulnerable to the Wang *et al.* differential attacks [52]. However, such attacks are successful on this family of hash functions due to the ability of the attacker to manipulate the propagation of differences in the internal state through message modification techniques, which are effective because the algorithm allows a user to feed the state with independent

message blocks for a substantial number of rounds. Nevertheless, without the message feeding algorithm, the Wang *et al.* attacks are ineffective and sLiSCP is an unkeyed permutation where the attacker has no means to manipulate the value of the internal state amid execution.

- *Simeck operations are bit-based and thus, can be used directly on a large state for the design of a permutation.* Simeck with large internal states such as Simeck-128 are hard to analyze, even the probabilities of their differential and linear characteristics are harder to bound for an extended number of rounds [41, 44]. Thus, providing the security guarantee for Simeck with a large state size is almost unpractical. Consequently, the adoption of Simeck-48 and Simeck-64 in a Type-2 GFS construction enables us to leverage their existing cryptanalysis and further provide bounds on the probabilities of differential and linear characteristics for the sLiSCP permutations.

7 Acknowledgment

The authors would like to thank the reviewers for their valuable comments that helped improve the quality of the paper. This work is supported by the Natural Sciences and Engineering Research Council of Canada (NSERC) and the National Institute of Standards and Technology (NIST).

References

- [1] ABED, F., LIST, E., LUCKS, S., AND WENZEL, J.: Differential cryptanalysis of round-reduced simon and speck. In: Cid, C., Rechberger, C. (Eds.), FSE 2014. LNCS, vol. 8540, pp. 525-545. Springer, Heidelberg (2015)
- [2] AGREN, M., HELL, M., JOHANSSON, T., AND MEIER, W.: Grain-128a: A new version of grain-128 with optional authentication. *Int. J. Wire. Mob. Comput.* 5(1), pp. 48-59. (2011)
- [3] ALTAWY, R., ROHIT, R., HE, M., MANDAL, K., YANG, G., AND GONG, G.: sLiSCP: Simeck-based permutations for lightweight sponge cryptographic primitives. The University of Waterloo CACR Archive, Technical Report CACR 2017-04, 2017. <http://cacr.uwaterloo.ca/>
- [4] ARMKNECHT, F., HAMANN, M., AND MIKHALEV, V.: Lightweight authentication protocols on ultra-constrained rfids - myths and facts. In: Saxena, N., Sadeghi, A.-R. (Eds.), RFIDSec 2014, Revised Selected Papers. LNCS, vol. 8651, pp. 1-18. Springer, Cham (2014)
- [5] AUMASSON, J.-P., HENZEN, L., MEIER, W., AND NAYA-PLASENCIA, M.: Quark: A lightweight hash. *Journal of Cryptology* 26(2), pp. 313-339. (2013)
- [6] AUMASSON, J.-P., JOVANOVIC, P., AND NEVES, S.: NORX: Parallel and scalable aead. In: M. Kutyłowski and J. Vaidya (Eds.), ESORICS 2014. LNCS, vol. 8713, pp. 19-36. Springer, Cham (2014)
- [7] AUMASSON, J.-P., JOVANOVIC, P., AND NEVES, S.: Norx8 and norx16: Authenticated encryption for low-end systems. *Cryptology ePrint Archive*, Report 2015/1154, 2015. <http://eprint.iacr.org/2015/1154>.
- [8] BABBAE, S., AND DODD, M.: The MICKEY stream ciphers. In: New stream cipher designs. LNCS, vol. 4986, pp. 191-209. Springer, Heidelberg (2008)
- [9] BEAULIEU, R., SHORS, D., SMITH, J., TREATMAN-CLARK, S., WEEKS, B., AND WINGERS, L.: The simon and speck families of lightweight block ciphers. *Cryptology ePrint Archive*, Report 2013/404, 2013. <http://eprint.iacr.org/2013/404>.

- [10] BEIERLE, C., JEAN, J., KÖLBL, S., LEANDER, G., MORADI, A., PEYRIN, T., SASAKI, Y., SASDRICH, P., AND SIM, S. M.: The skinny family of block ciphers and its low-latency variant MANTIS. In: M. Robshaw and J. Katz (Eds.), CRYPTO 2016. LNCS, vol. 9815, pp. 123–153. Springer, Heidelberg (2016)
- [11] BERTONI, G., DAEMEN, J., PEETERS, M., AND ASSCHE, G.: Caesar submission: Ketje v2, 2014. <http://ketje.noekeon.org/Ketje2-doc2.0.pdf>.
- [12] BERTONI, G., DAEMEN, J., PEETERS, M., AND VAN ASSCHE, G.: On the indistinguishability of the sponge construction. In: N. Smart (Ed.), EUROCRYPT 2008. LNCS, vol. 4965, pp. 181–197. Springer, Heidelberg (2008)
- [13] BERTONI, G., DAEMEN, J., PEETERS, M., AND VAN ASSCHE, G.: Keccak specifications. Submission to nist (round 2).(2009)
- [14] BERTONI, G., DAEMEN, J., PEETERS, M., AND VAN ASSCHE, G.: On the security of the keyed sponge construction. In: Symmetric Key Encryption Workshop 2011.
- [15] BERTONI, G., DAEMEN, J., PEETERS, M., AND VAN ASSCHE, G.: Duplexing the sponge: Single-pass authenticated encryption and other applications. In: A. Miri and S. Vaudenay (Eds.), SAC 2012. LNCS, vol. 7118, pp. 320–337. Springer, Heidelberg (2012)
- [16] BERTONI, G., DAEMEN, J., PEETERS, M., AND VAN ASSCHE, G.: Permutation-based encryption, authentication and authenticated encryption. In: DIAC 2012.
- [17] BERTONI, G., DAEMEN, J., PEETERS, M., AND VAN ASSCHE, G.: Cryptographic sponge functions, 2014. <http://sponge.noekeon.org/CSF-0.1.pdf>.
- [18] BIHAM, E., BIRYUKOV, A., AND SHAMIR, A.: Cryptanalysis of skipjack reduced to 31 rounds using impossible differentials. In: J. Stern, (Ed.), EUROCRYPT 1999. LNCS, vol. 1592, pp. 12–23. Springer, Heidelberg (1999)
- [19] BIRYUKOV, A., ROY, A., AND VELICHKOV, V.: Differential analysis of block ciphers simon and speck. In: C. Cid and C. Rechberger (Eds.), FSE 2014. LNCS, vol. 8540, pp. 546–570. Springer, Heidelberg (2014)
- [20] BIRYUKOV, A., AND WAGNER, D.: Slide attacks. In: L. Knudsen (Ed.), FSE 1999. LNCS, vol. 1636, pp. 245–259. Springer, Heidelberg (1999)
- [21] BLONDEAU, C., BOGDANOV, A., AND WANG, M.: On the (in)equivalence of impossible differential and zero-correlation distinguishers for feistel- and skipjack-type ciphers. In: I. Boureanu, P. Owesarski, and S. Vaudenay (Eds.), ACNS 2014. LNCS, vol. 8479, pp. 271–288. Springer, Cham (2014)
- [22] BLONDEAU, C., AND MINIER, M.: Analysis of impossible, integral and zero-correlation attacks on type-ii generalized feistel networks using the matrix method. newblock In: G. Leander (Ed.), FSE 2015. LNCS, vol. 9054, pp. 92–113. Springer, Heidelberg (2015)
- [23] BOGDANOV, A., KNEŽEVIĆ, M., LEANDER, G., TOZ, D., VARICI, K., AND VERBAUWHEDE, I.: Spongint: A lightweight hash function. In: B. Preneel and T. Takagi (Eds.), CHES 2011. LNCS, vol. 6917, pp. 312–325. Springer, Heidelberg (2011)
- [24] BOGDANOV, A., KNUDSEN, L. R., LEANDER, G., PAAR, C., POSCHMANN, A., ROBshaw, M. J. B., SEURIN, Y., AND VIKKELSOE, C.: PRESENT: An ultra-lightweight block cipher. In: P. Paillier and I. Verbauwhede (Eds.), CHES 2007. LNCS, vol. 4727, pp. 450–466. Springer, Heidelberg (2007)
- [25] BOGDANOV, A., LEANDER, G., NYBERG, K., AND WANG, M.: Integral and multi-dimensional linear distinguishers with correlation zero. In: X. Wang and K. Sako, (Eds.), ASIACRYPT 2012. LNCS, vol. 7658, pp. 244–261. Springer, Heidelberg (2012)
- [26] BOGDANOV, A., AND SHIBUTANI, K.: Generalized feistel networks revisited. Designs, Codes and Cryptography 66(1), pp. 75–97. (2013)

- [27] BORGHOFF, J., CANTEAUT, A., GÜNEYSU, T., KAVUN, E. B., KNEZEVIĆ, M., KNUDSEN, L. R., LEANDER, G., NIKOV, V., PAAR, C., RECHBERGER, C., ROMBOUTS, P., THOMSEN, S. S., AND YALÇIN, T.: PRINCE – A low-latency block cipher for pervasive computing applications. In: Wang X., Sako K. (Eds), ASIACRYPT 2012. LNCS, vol. 7658, pp. 208–225. Springer, Heidelberg (2012)
- [28] CAESAR: Competition for authenticated encryption: security, applicability, and robustness. <https://competitions.cr.yp.to/caesar.html>.
- [29] DE CANNIÈRE, C.: Trivium: A stream cipher construction inspired by block cipher design principles. In: Katsikas S.K., Lopez J., Backes M., Gritzalis S., Preneel B.(Eds.), Information Security 2006. LNCS, vol. 4176, , pp. 171–186. Springer, Heidelberg (2006)
- [30] DE CANNIÈRE, C., DUNKELMAN, O., AND KNEŽEVIĆ, M.: KATAN and KTANTAN — A family of small and efficient hardware-oriented block ciphers. In: Clavier C., Gaj K. (Eds.), CHES 2009. LNCS, vol. 5747, pp. 272–288. Springer, Heidelberg (2009)
- [31] DINU, D., PERRIN, L., UDOVENKO, A., VELICHKOV, V., GROSSSCHÄDL, J., AND BIRYUKOV, A.: Design strategies for arx with provable bounds: Sparx and lax. In: J. H. Cheon and T. Takagi (Eds.), ASIACRYPT 2016. LNCS, vol. 10031, pp. 484–513. Springer, Heidelberg (2016)
- [32] DOBRAUNIG, C., EICHLSEDER, M., MENDEL, F., AND SCHLÄFFER, M.: Ascon v1.2. Submission to the caesar competition: <http://competitions.cr.yp.to/round3/asconv12.pdf>, 2016.
- [33] GUERON, S., AND MOUHA, N.: Simpira v2: A family of efficient permutations using the aes round function. In: J. H. Cheon and T. Takagi (Eds.), ASIACRYPT 2016. LNCS, vol. 10031, pp. 95–125. Springer, Heidelberg (2016)
- [34] GUO, J., PEYRIN, T., AND POSCHMANN, A.: The photon family of lightweight hash functions. In: P. Rogaway (Ed.), CRYPTO 2011. LNCS, vol. 6841, pp. 222–239. Springer, Heidelberg (2011)
- [35] GUO, J., PEYRIN, T., POSCHMANN, A., AND ROBshaw, M.: The led block cipher. In: B. Preneel and T. Takagi (Eds.), CHES 2011. LNCS, vol. 6917, pp. 326–341. Springer, Heidelberg (2011)
- [36] HELL, M., JOHANSSON, T., MAXIMOV, A., AND MEIER, W.: A stream cipher proposal: Grain-128. In: IEEE International Symposium on Information Theory, pp. 1614–1618. (2006)
- [37] HONG, D., SUNG, J., HONG, S., LIM, J., LEE, S., KOO, B.-S., LEE, C., CHANG, D., LEE, J., JEONG, K., KIM, H., KIM, J., AND CHEE, S.: HIGHT: A new block cipher suitable for low-resource device. In: Goubin L., Matsui M (Eds.), CHES 2006. LNCS, vol. 4249, pp. 46–59. Springer, Heidelberg (2006)
- [38] JUELS, A., AND WEIS, S. A.: Authenticating pervasive devices with human protocols. In: Shoup V. (Ed.), CRYPTO 2005. LNCS, vol. 3621, pp. 293–308. Springer, Heidelberg (2005)
- [39] KAVUN, E. B., AND YALCIN, T.: A lightweight implementation of keccak hash function for radio-frequency identification applications. In: Ors Yalcin S.B (Ed.), RFIDSec 2010. LNCS, vol. 6370, pp. 258–269. Springer, Heidelberg (2010)
- [40] KELIHER, L.: Exact maximum expected differential and linear probability for two-round advanced encryption standard. IET Information Security 1 , pp. 53–57(4). (2007)
- [41] KÖLBL, S., LEANDER, G., AND TIESSEN, T.: Observations on the simon block cipher family. In: R. Gennaro and M. Robshaw (Eds.), CRYPTO 2015. LNCS, vol. 9215, pp. 258–269. Springer, Heidelberg (2015)

- [42] KONDO, K., SASAKI, Y., AND IWATA, T.: On the design rationale of simon block cipher: Integral attacks and impossible differential attacks against simon variants. In: M. Manulis, A.-R. Sadeghi, and S. Schneider (Eds.), ACNS 2016. LNCS, vol. 9696, pp. 518–536. Springer, CHAM (2016)
- [43] LEANDER, G., ABDELRAHEEM, M. A., ALKHZAIMI, H., AND ZENNER, E.: A cryptanalysis of printcipher: The invariant subspace attack. In: P. Rogaway (Ed.), CRYPTO 2011. LNCS, vol. 6841, pp. 206–221. Springer, Heidelberg (2011)
- [44] LIU, Z., LI, Y., AND WANG, M.: Optimal differential trails in simon-like ciphers. IACR TOSC 2017, pp. 358–379. (2017)
- [45] MCKAY, K., BASSHAM, L., SÖNMEZ TURAN, M., AND MOUHA, N.: Report on lightweight cryptography (NISTIR8114). (2017)
- [46] NAWAZ, Y., AND GONG, G.: Wg: A family of stream ciphers with designed randomness properties. Inf. Sci. 178(7), pp. 1903–1916. (2008)
- [47] NYBERG, K.: Generalized feistel networks. In: K. Kim and T. Matsumoto (Eds.), ASIACRYPT 1996. LNCS, vol. 1163, pp. 91–104. Springer, Heidelberg (2011)
- [48] RØNJOM, S.: Invariant subspaces in simpira. Cryptology ePrint Archive, Report 2016/248, 2016. <http://eprint.iacr.org/2016/248>.
- [49] SUZAKI, T., MINEMATSU, K., MORIOKA, S., AND KOBAYASHI, E.: Twine: A lightweight block cipher for multiple platforms. In: Knudsen L.R., Wu H. (Eds.), SAC 2012. LNCS, vol. 7707, pp. 339–354. Springer, Heidelberg (2012)
- [50] TODO, Y.: Structural evaluation by generalized integral property. In: E. Oswald and M. Fischlin (Eds.), EUROCRYPT 2015. LNCS, vol. 9056, pp. 287–314. Springer, Heidelberg (2015)
- [51] WANG, Q., LIU, Z., VARICI, K., SASAKI, Y., RIJMEN, V., AND TODO, Y.: Cryptanalysis of reduced-round simon32 and simon48. In: W. Meier and D. Mukhopadhyay (Eds.) INDOCRYPT 2014. LNCS, vol. 8885, pp. 143–160. Springer, Cham (2014)
- [52] WANG, X., AND YU, H.: How to break md5 and other hash functions. In: R. Cramer (Ed.), EUROCRYPT 2005. LNCS, vol. 3494, pp. 19–35. Springer, Heidelberg (2005)
- [53] WHEELER, D., AND NEEDHAM, R.: Tea, a tiny encryption algorithm. In: Preneel B. (Ed.), FSE 1994. LNCS, vol. 1008, pp. 97–110. Springer, Heidelberg (2005)
- [54] XIANG, Z., ZHANG, W., BAO, Z., AND LIN, D.: Applying milp method to searching integral distinguishers based on division property for 6 lightweight block ciphers. In: J. H. Cheon and T. Takagi (Eds.), ASIACRYPT 2016. LNCS, vol. 10031, pp. 648–678. Springer, Heidelberg (2016)
- [55] YANG, G., ZHU, B., SUDER, V., AAGAARD, M. D., AND GONG, G.: The simeck family of lightweight block ciphers. In: T. Güneysu and H. Handschuh (Eds.), CHES 2015. LNCS, vol. 9293, pp. 307–329. Springer, Heidelberg (2015)
- [56] YAP, H., KHOO, K., POSCHMANN, A., AND HENRICKSEN, M.: EPCBC - A block cipher suitable for electronic product code encryption. In: Lin D., Tsudik G., Wang X.(Eds.), CANS 2011. LNCS, vol. 7092, pp. 76–97. Springer, Heidelberg (2011)
- [57] ZHANG, H., AND WU, W.: Structural evaluation for generalized feistel structures and applications to lblock and twine. In: A. Biryukov and V. Goyal (Eds.), INDOCRYPT 2015. LNCS, vol. 9462, pp. 218–237. Springer, Cham (2015)